

# Case 5: Zero-to-Deploy

## Production-Ready Task Management API

Your Name

May 12, 2026

# Problem Statement

## Objective

Take a small backend service and make it production-ready using modern DevOps practices.

## Key Goals

- Automated deployment
- CI/CD pipeline
- Docker containerization
- Monitoring and logging
- Reliable cloud deployment

## Focus Area

Operational maturity over application complexity.

# Project Overview

## Project Built

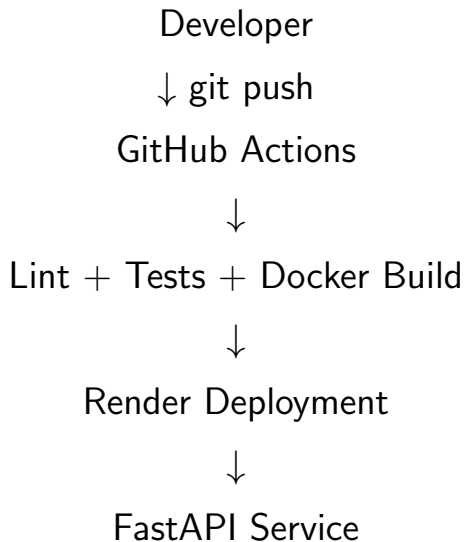
Task Management REST API

## Core Features

- Create tasks
- Update tasks
- Delete tasks
- Retrieve tasks
- Health monitoring endpoint

## API Endpoints

- GET /health
- GET /tasks
- POST /tasks
- PUT /tasks/{id}



# Technology Stack

| Layer            | Technology        |
|------------------|-------------------|
| Backend          | FastAPI           |
| Database         | PostgreSQL (Neon) |
| ORM              | SQLAlchemy        |
| Deployment       | Render            |
| Containerization | Docker            |
| CI/CD            | GitHub Actions    |
| Monitoring       | UptimeRobot       |
| Testing          | Pytest            |
| Logging          | Python Logging    |

# Docker Containerization

## Why Docker?

- Consistent deployments
- Reproducible environments
- Simplified cloud hosting
- Easy local development

## Key Features

- Multi-stage builds
- Lightweight image
- Health checks
- Layer caching optimization

## Docker Commands

```
docker build -t task-api .  
docker compose up -d
```

## Pipeline Workflow

- 1 Push code to GitHub
- 2 GitHub Actions starts
- 3 Linting executes
- 4 Tests run automatically
- 5 Docker image builds
- 6 Render deploys latest version

`git push` → automatic production update

# Deployment Setup

## Hosting Stack

- Render → application hosting
- Neon → PostgreSQL database
- GitHub → source control
- GitHub Actions → CI/CD
- UptimeRobot → monitoring

## Production Environment Variables

```
DATABASE_URL=postgresql://...  
SECRET_KEY=...  
ENVIRONMENT=production  
DEBUG=False
```



# Monitoring and Reliability

## Health Endpoint

```
GET /health
```

## Monitoring Includes

- API availability
- Database connectivity
- Response status
- Downtime alerts

## Logging

- Request logs
- Error logs
- Startup logs
- Deployment debugging

# Testing Strategy

## Testing Framework

Pytest

## Test Coverage

- Health endpoint tests
- CRUD API tests
- Database interaction tests

## Commands

```
pytest tests/ -v  
pytest tests/test_tasks.py
```

# Key Learnings

- Production deployment workflows
- CI/CD automation
- Docker-based deployments
- Health monitoring systems
- Operational reliability thinking
- Cloud infrastructure basics

## Main Takeaway

Building software is not only writing APIs. Deployment, monitoring, and reliability are equally important.

# Future Improvements

- JWT authentication
- Rate limiting
- Redis caching
- Prometheus + Grafana monitoring
- Distributed tracing
- Blue/Green deployment
- Centralized log aggregation

# Thank You

Questions?